

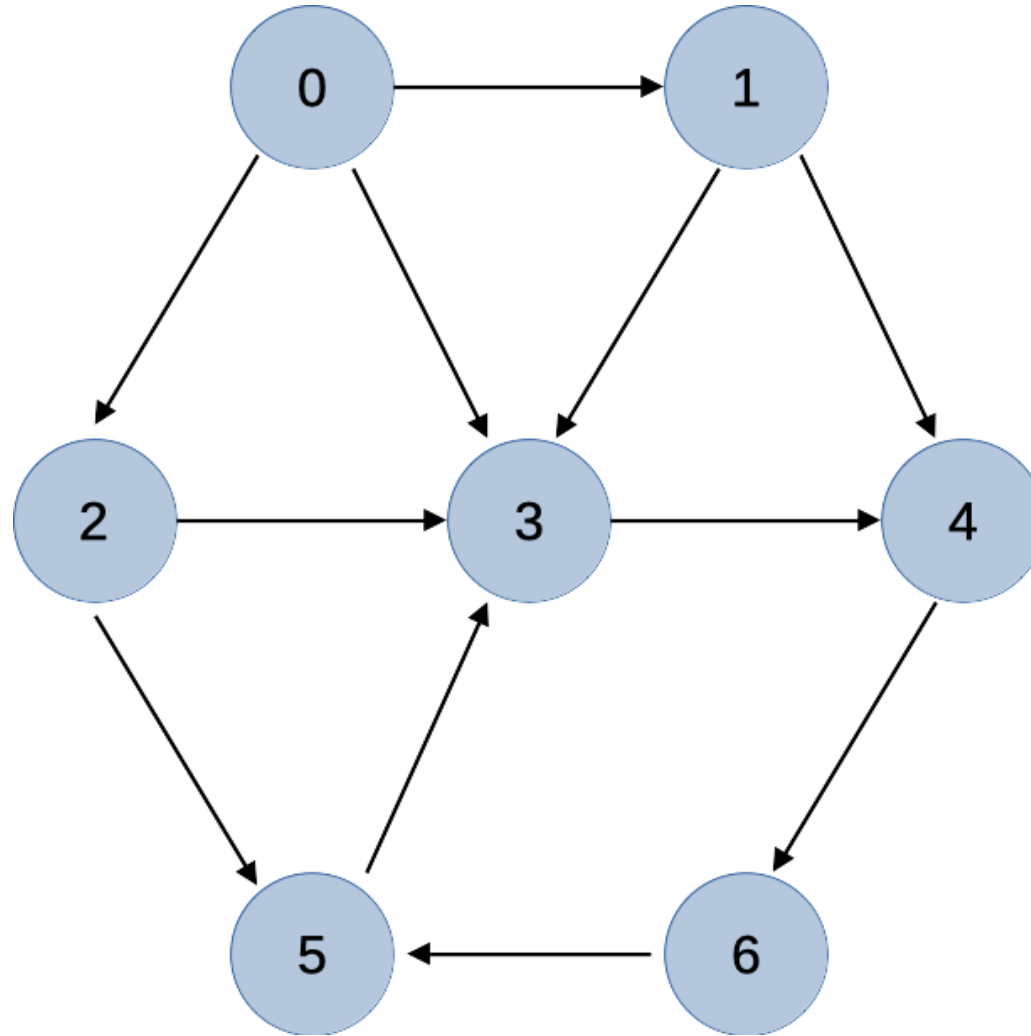
Competitive Programming

Graph Traversal

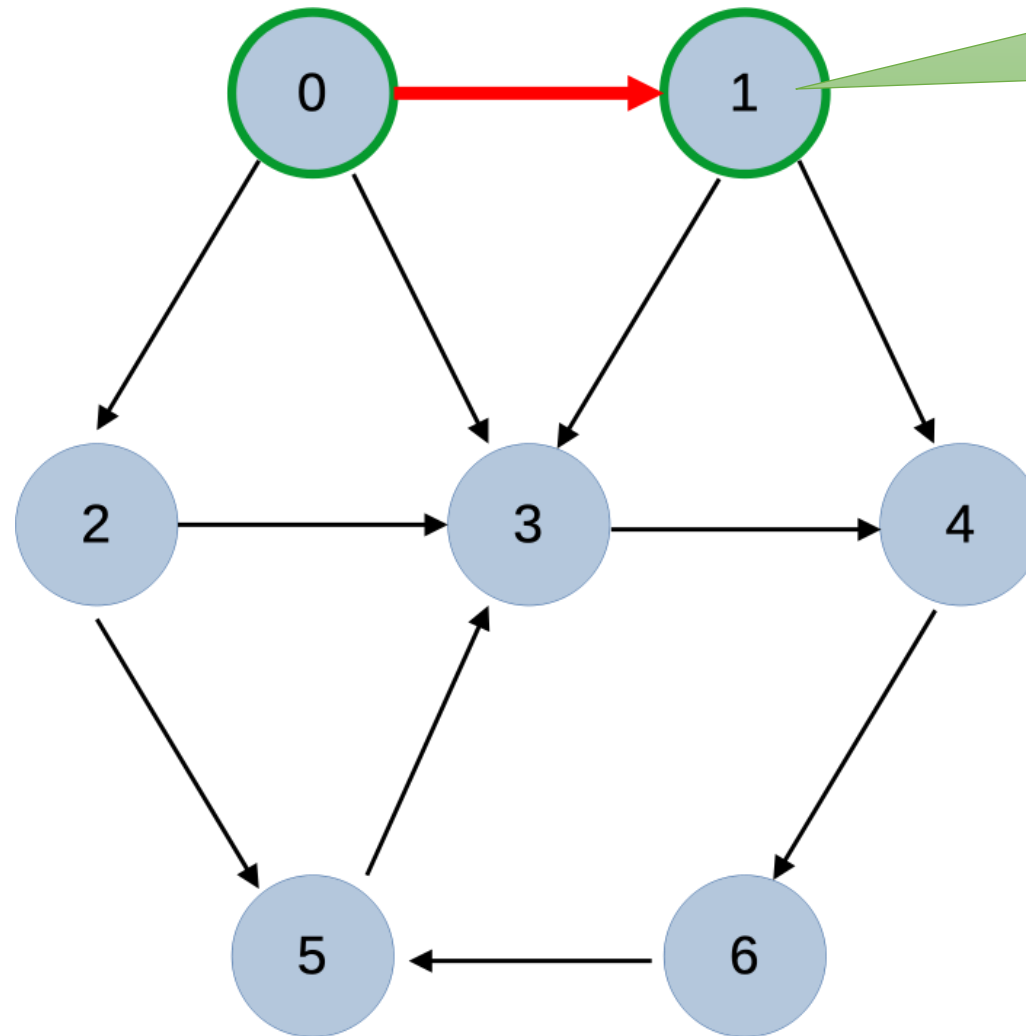
Graph Traversal

- Graph traversal : visit the vertices (in a particular order)
- Depth-first order
 - Easy to implement recursively
 - Good for finding cycles, finding components, topological-sort
- Breadth-first traversal
 - Easy-ish to implement with a queue
 - Good for finding shortest paths (measured in edges)

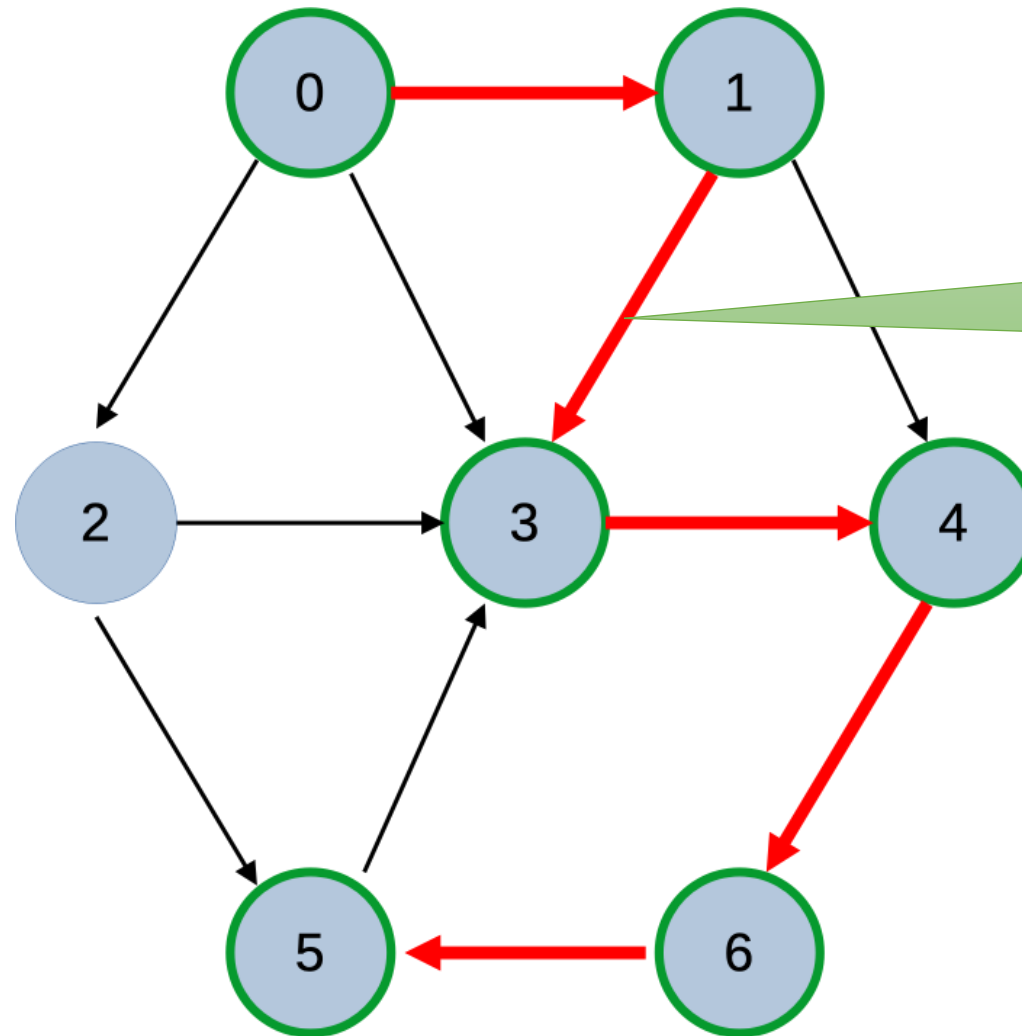
Example Graph



Depth-First Traversal

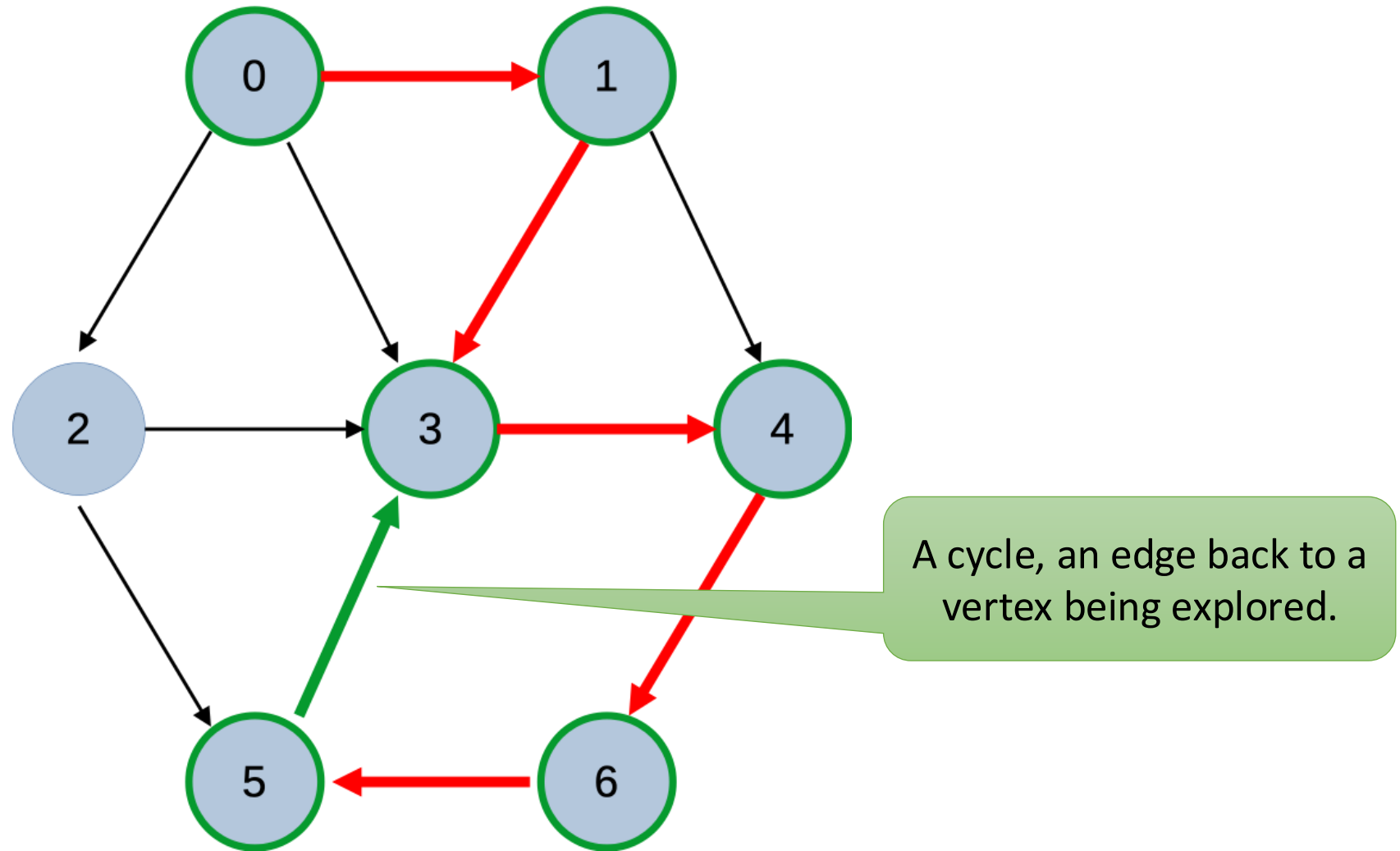


Depth-First Traversal

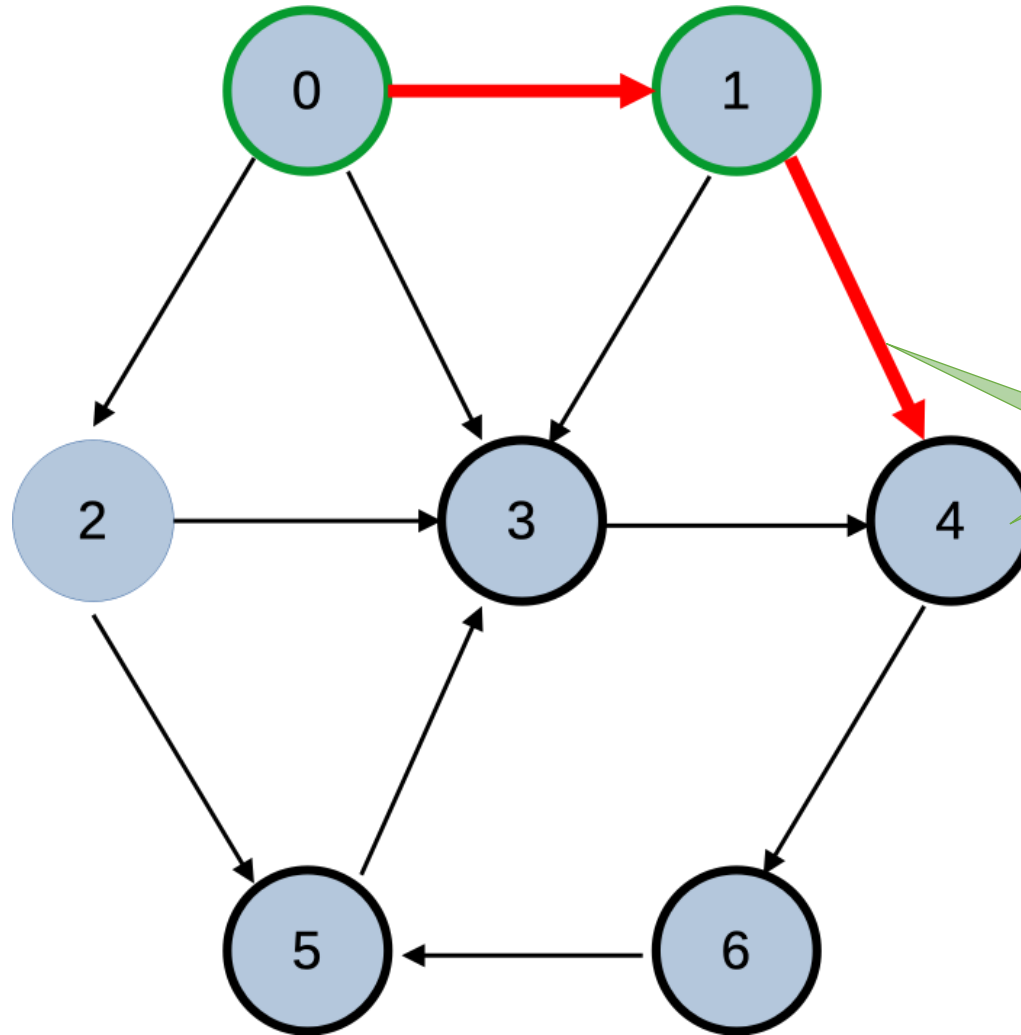


Following edges out of each vertex as we discover it.

Depth-First Traversal



Depth-First Traversal



Changing the color of a vertex as we finish exploring it.

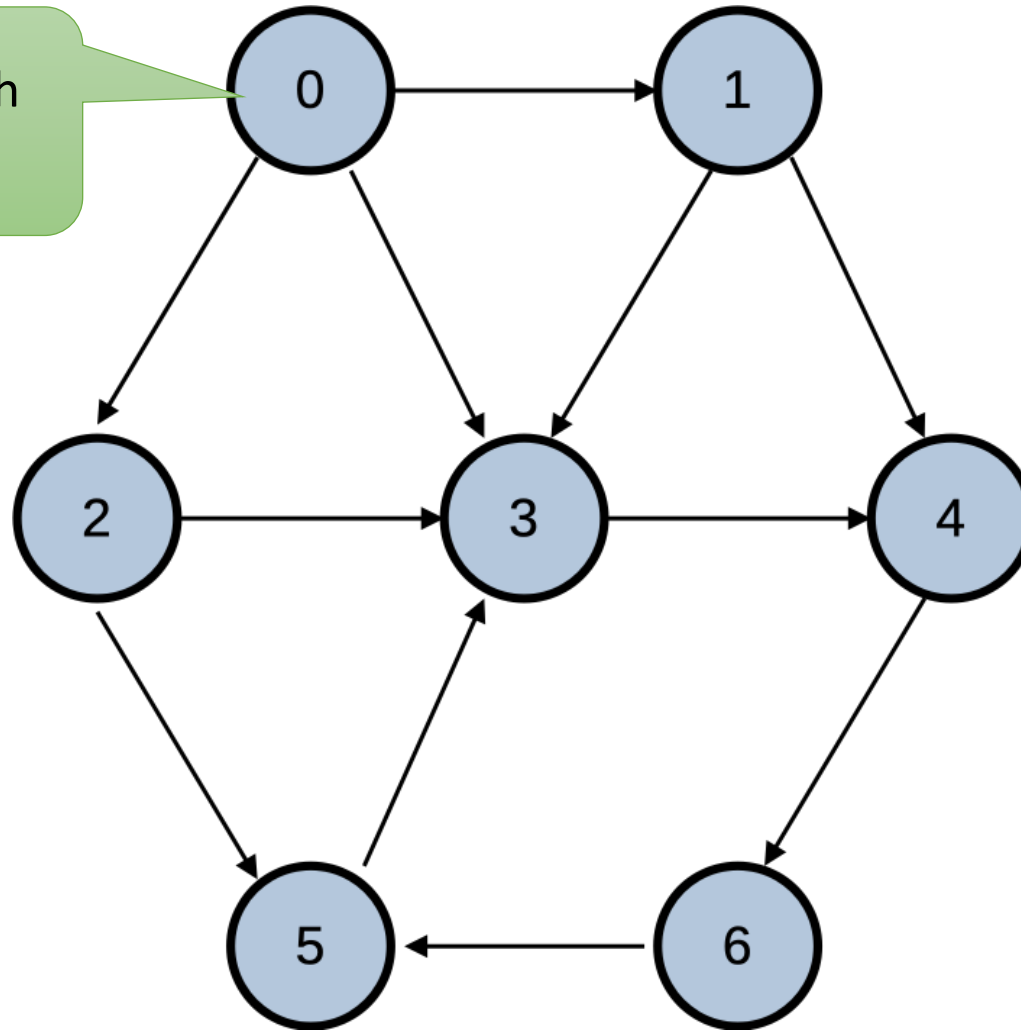
Prevents repeated exploring of the same part of the graph (exponential time).

Depth-First Traversal

```
void traverse( int i, int state[ n ] ) {  
    if ( state[ i ] == 2 )    // Are we finished with this vertex?  
        return;  
  
    if ( state[ i ] == 1 )    // Are we currently exploring this vertex?  
        // this is a cycle.  
        return;  
  
    state[ i ] = 1;           // OK.  Now, we're exploring from this vertex.  
  
    for ( j : neighbors[ i ] ) // Recursively explore its neighbors.  
        traverse( j, state );  
  
    state[ i ] = 2;           // Now, we're done with this vertex.  
}
```

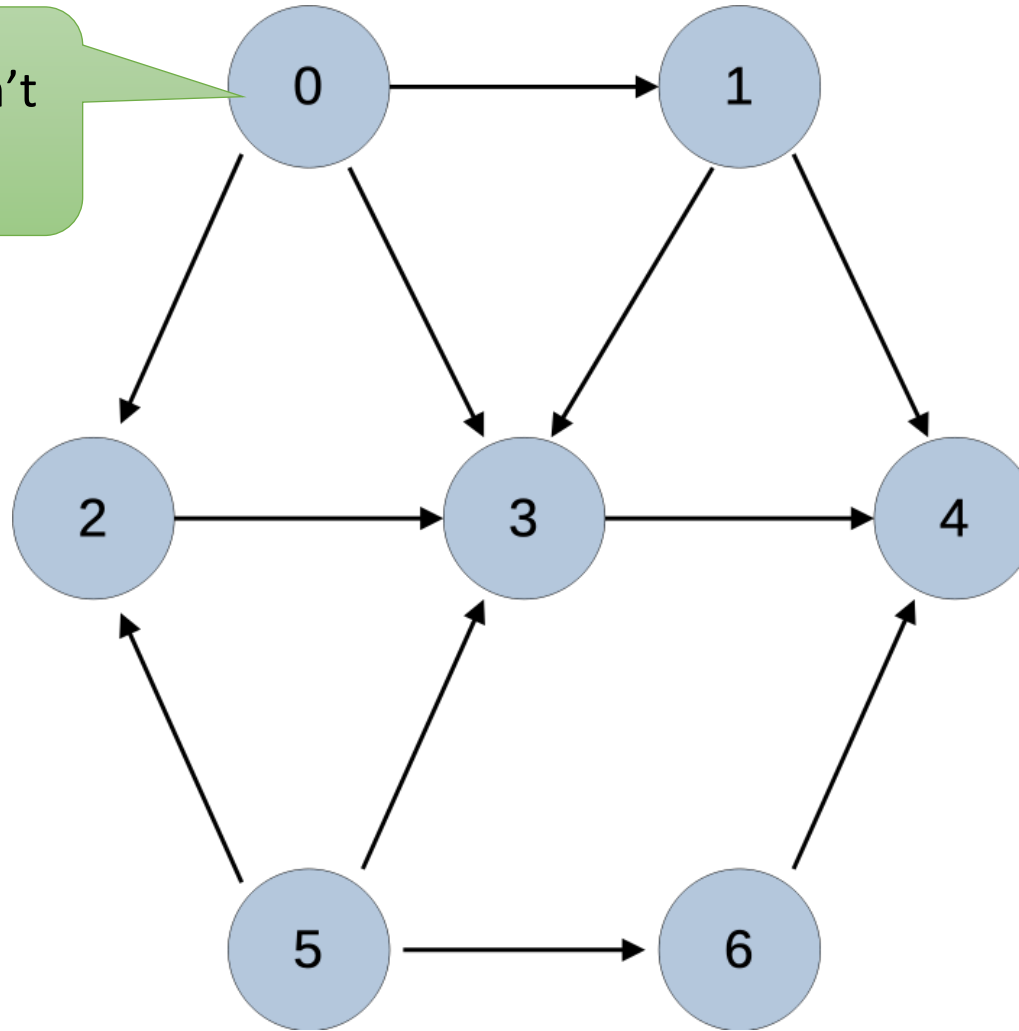

Depth-First Traversal

Starting here will reach every vertex.

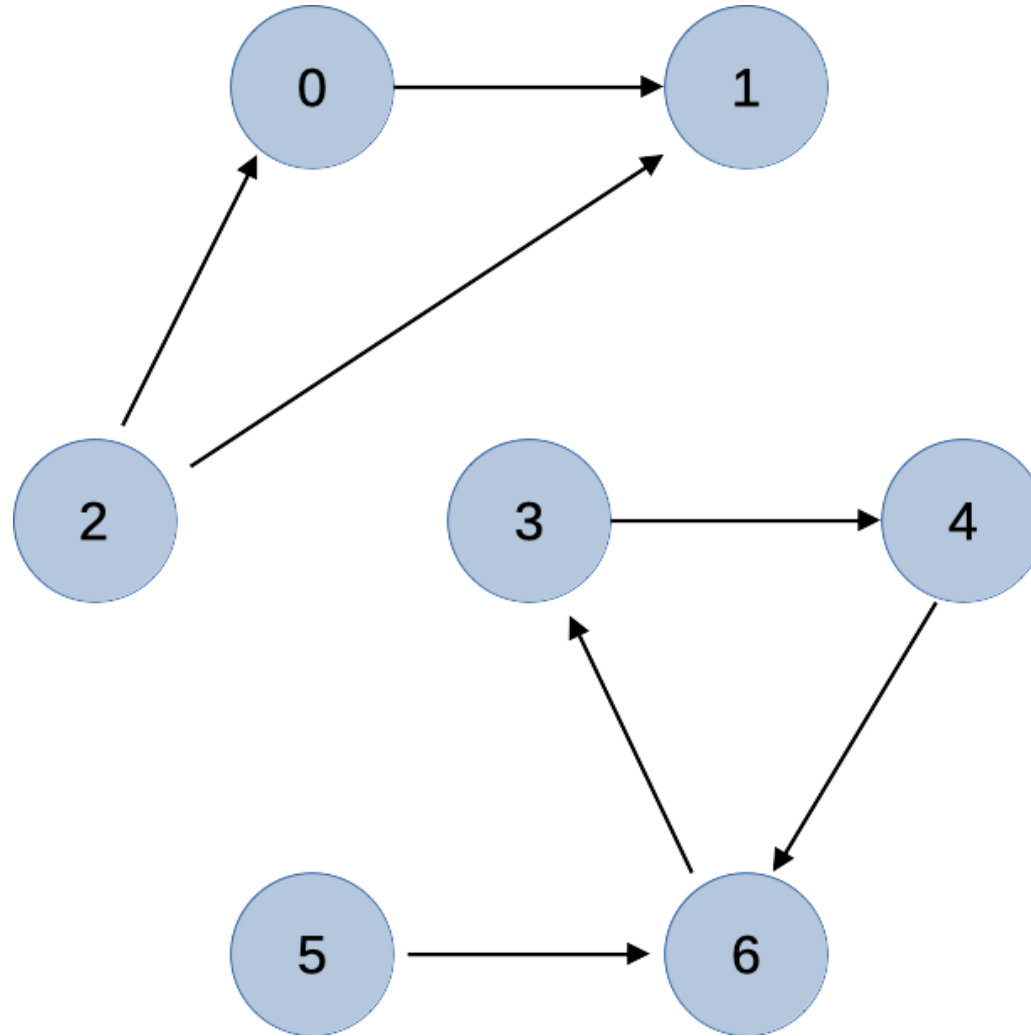


Where to start?

Now, starting here won't reach 5 and 6.



Multiple Components

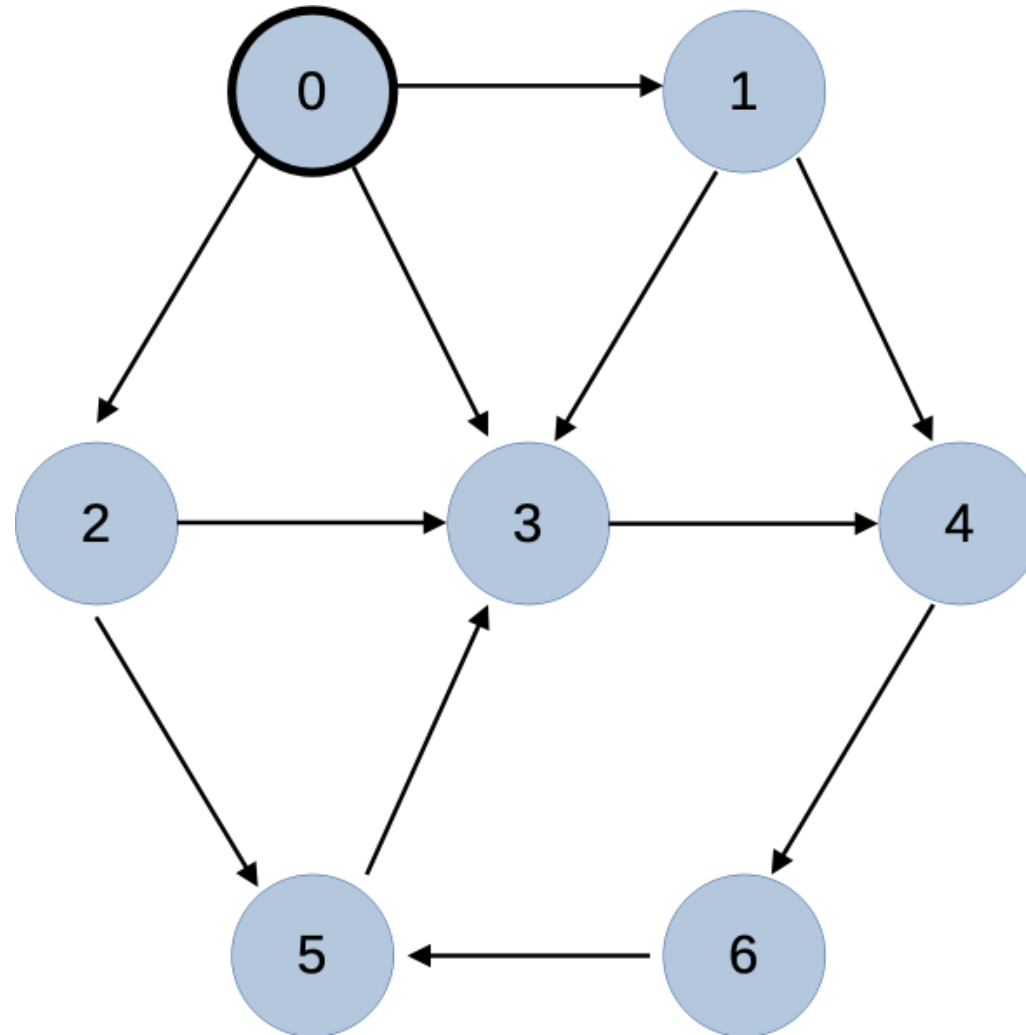


Visiting Multiple Components

- Reaching the whole graph may require multiple traversals.
- Maintaining the state will let us avoid duplicate traversal.

```
state = [ 0, 0, ... 0 ];    // Haven't explored any vertex yet.  
  
for ( i = 0; i < n; i++ )  // Try a traversal starting from any vertex.  
    traverse( i, state );
```

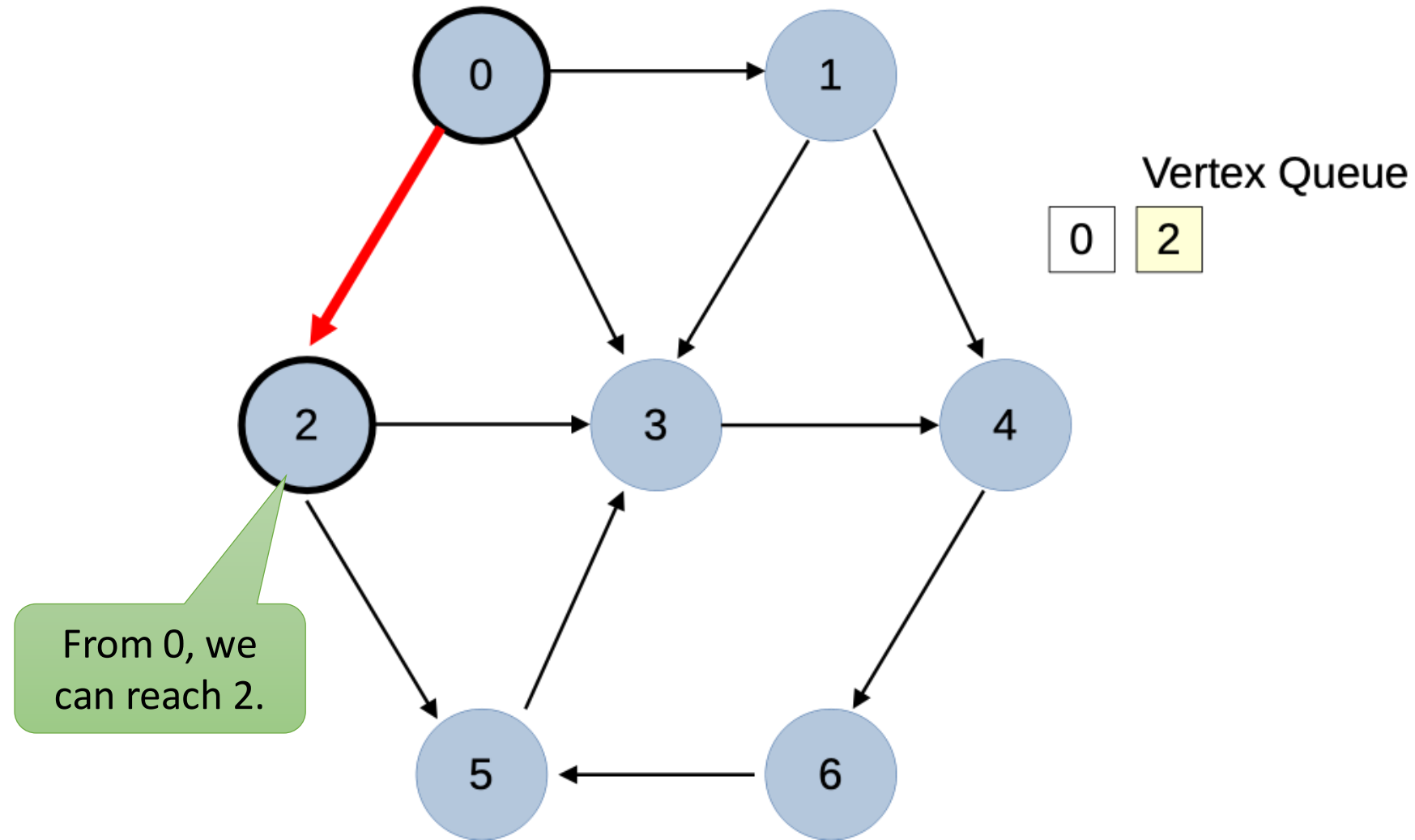
Breadth-First Order



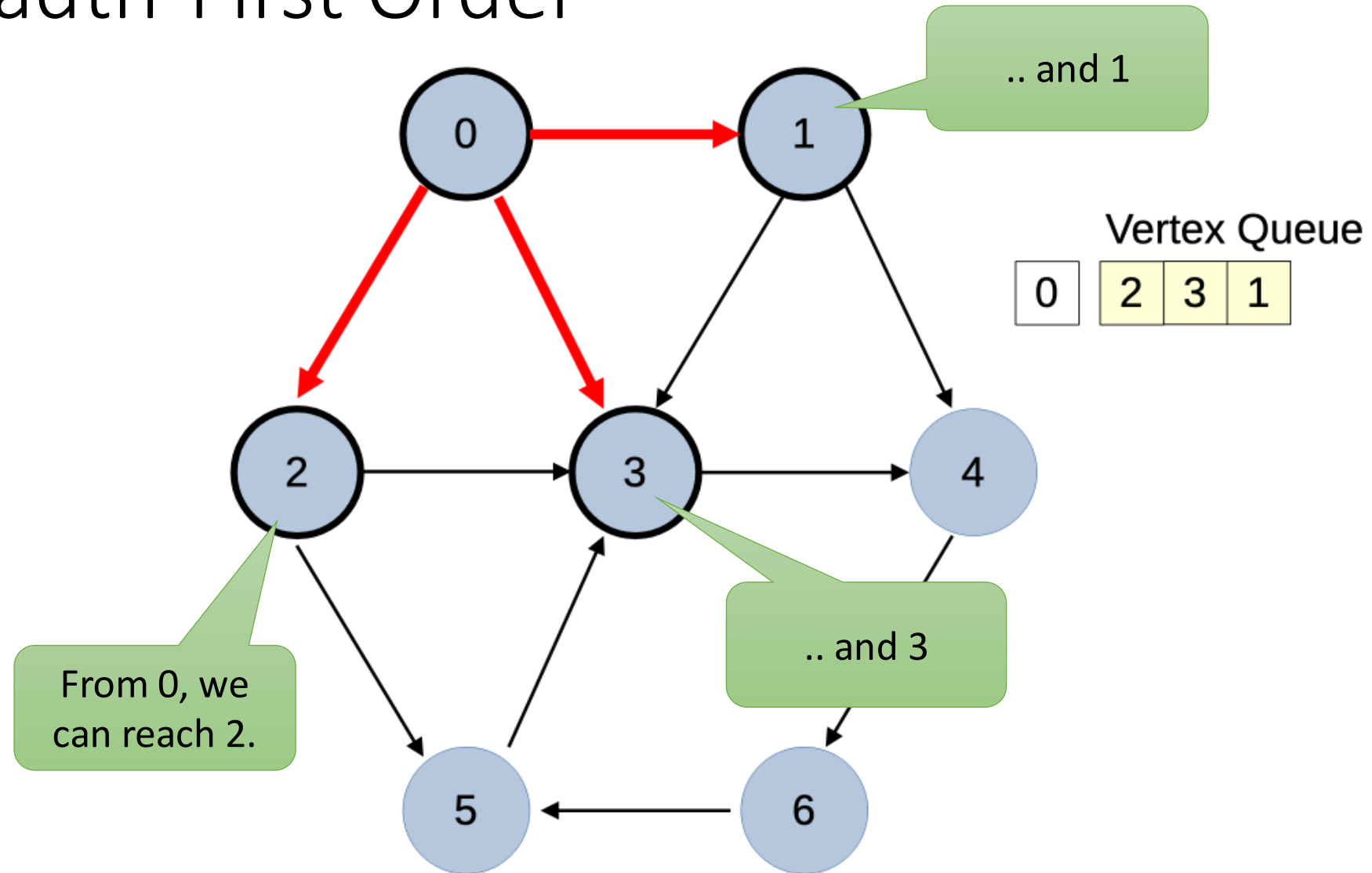
Vertex Queue

0

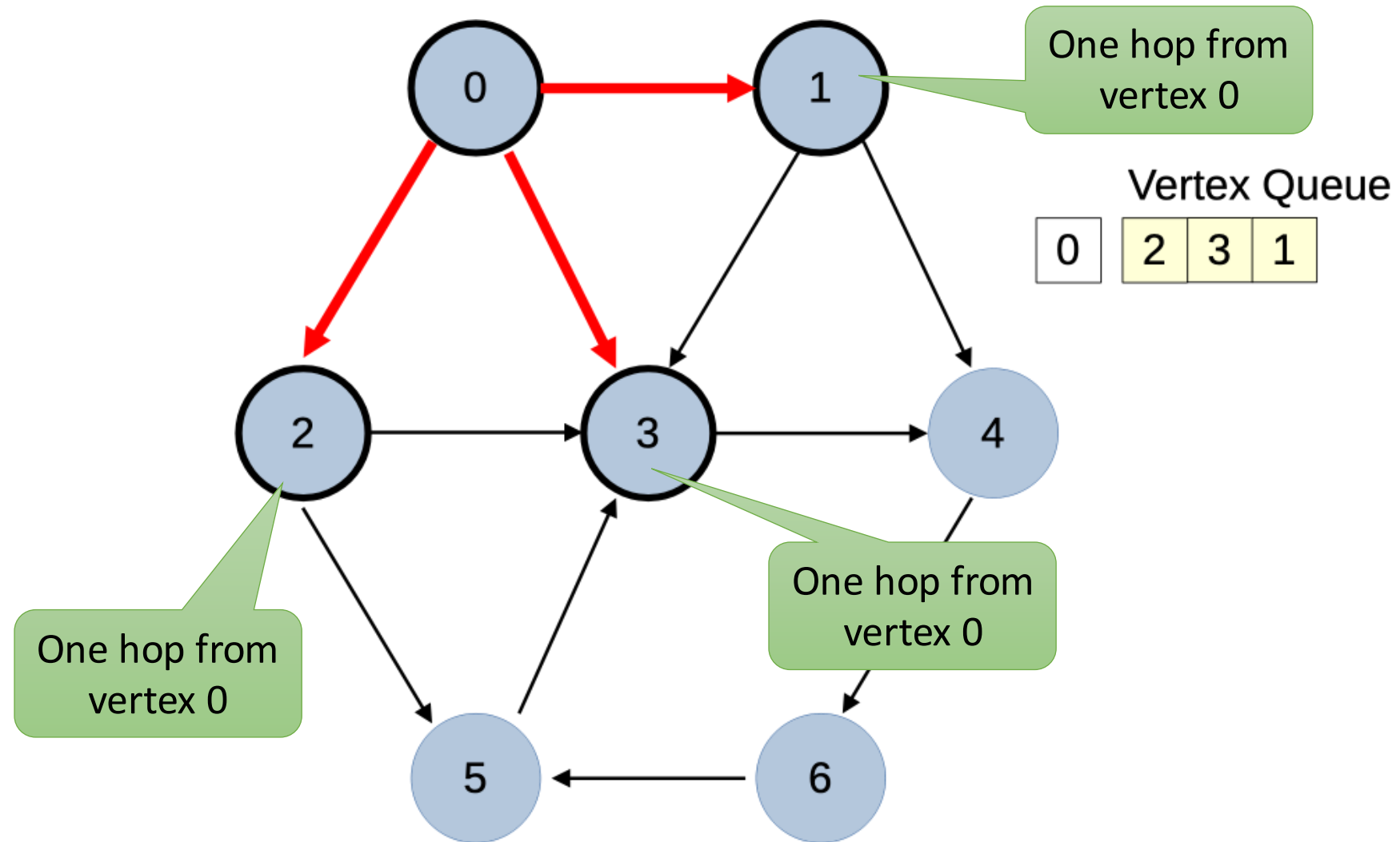
Breadth-First Order



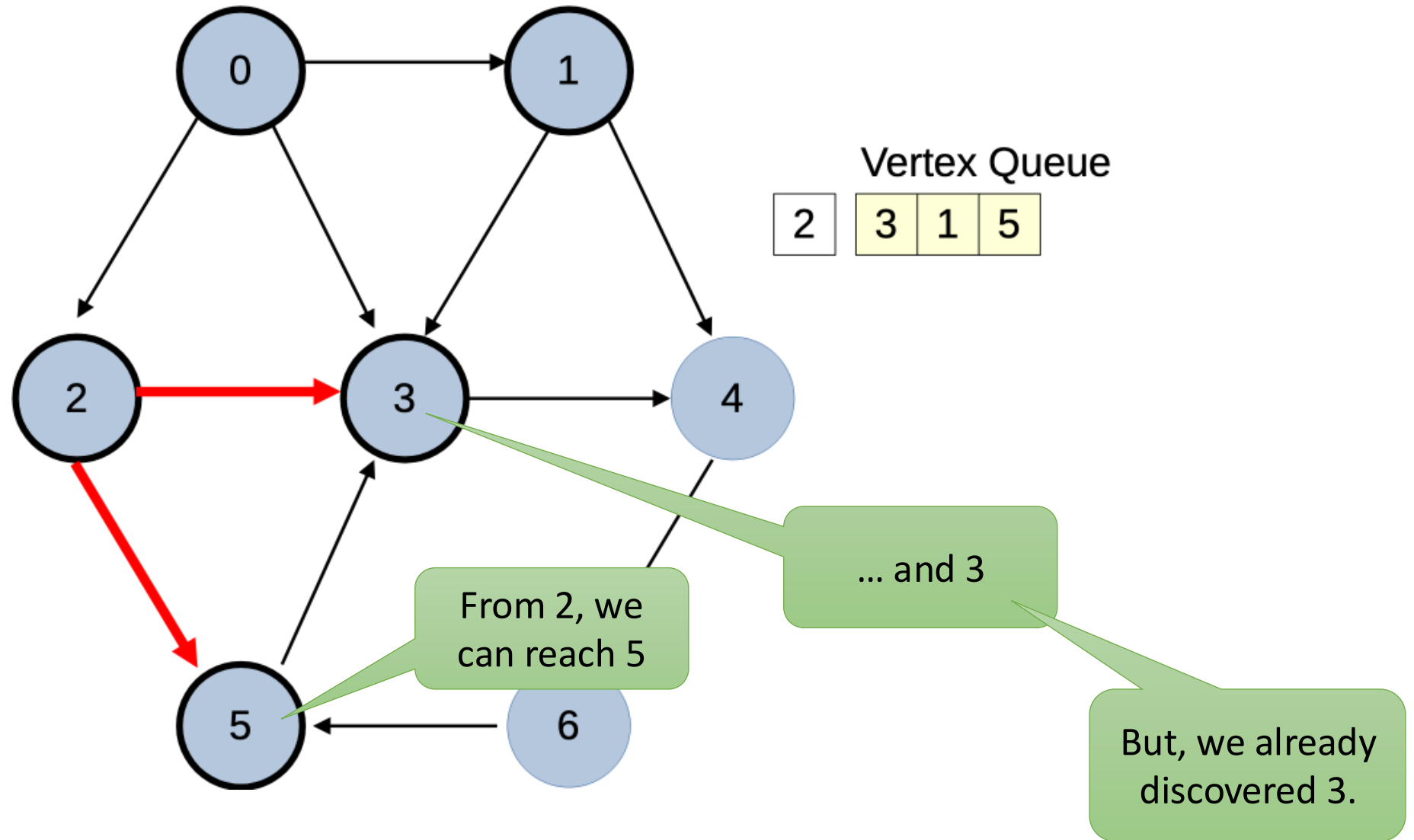
Breadth-First Order



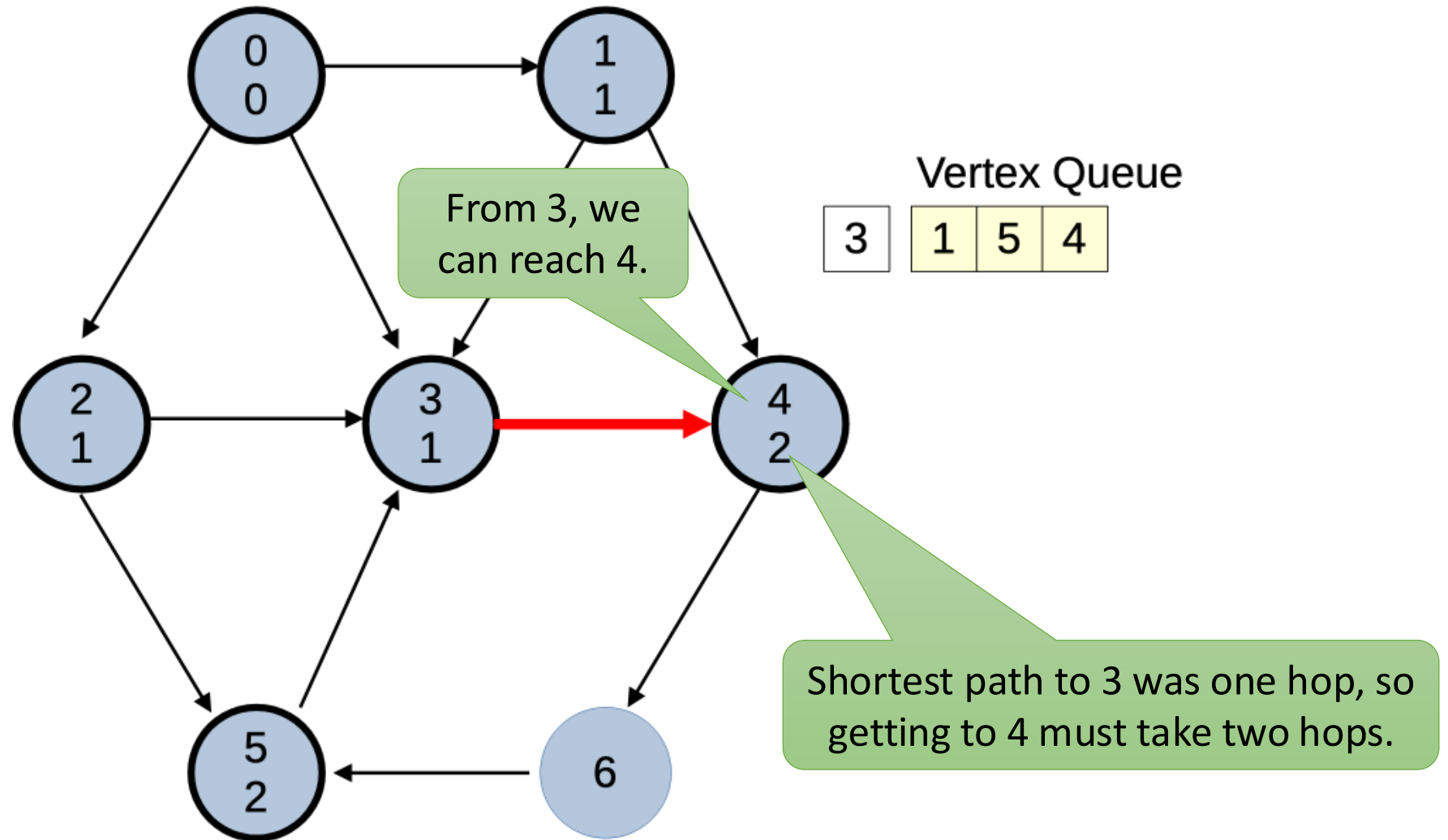
Breadth-First Order



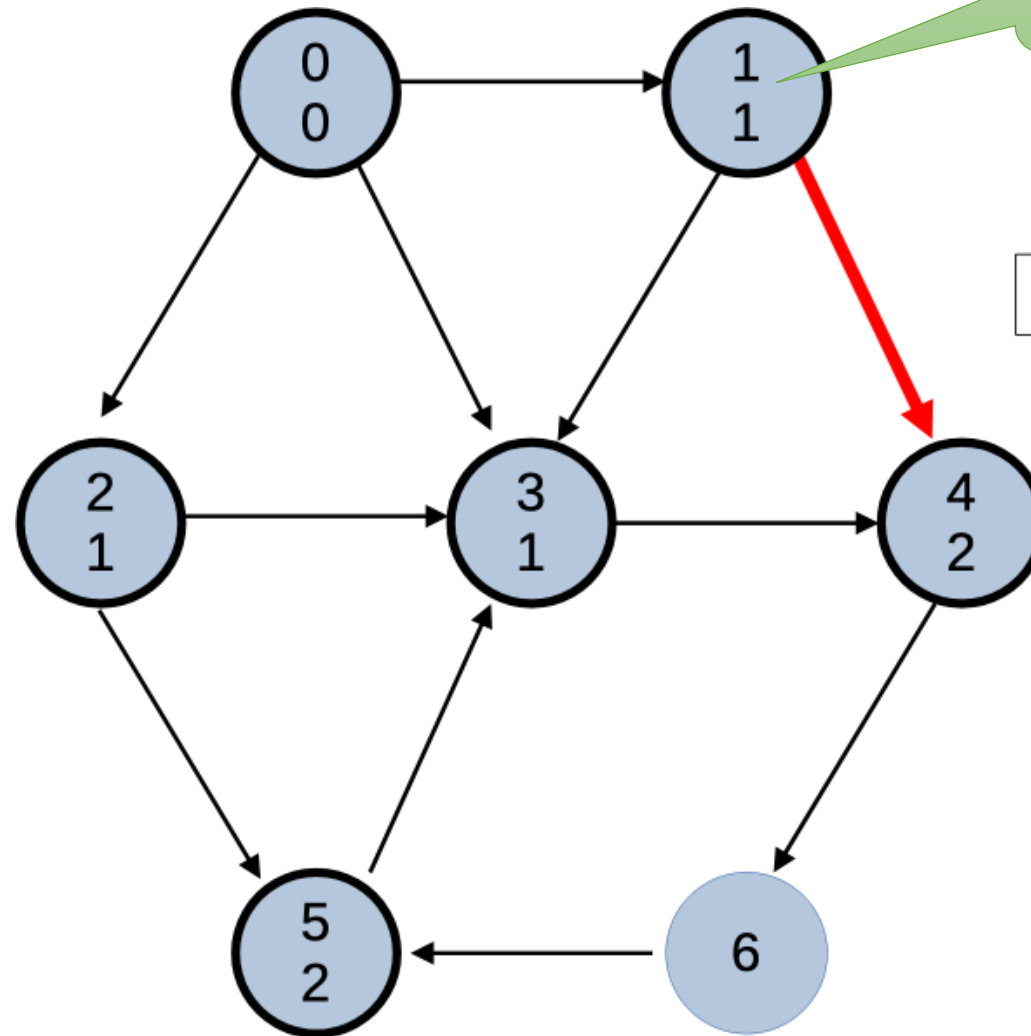
Breadth-First Order



Breadth-First Order

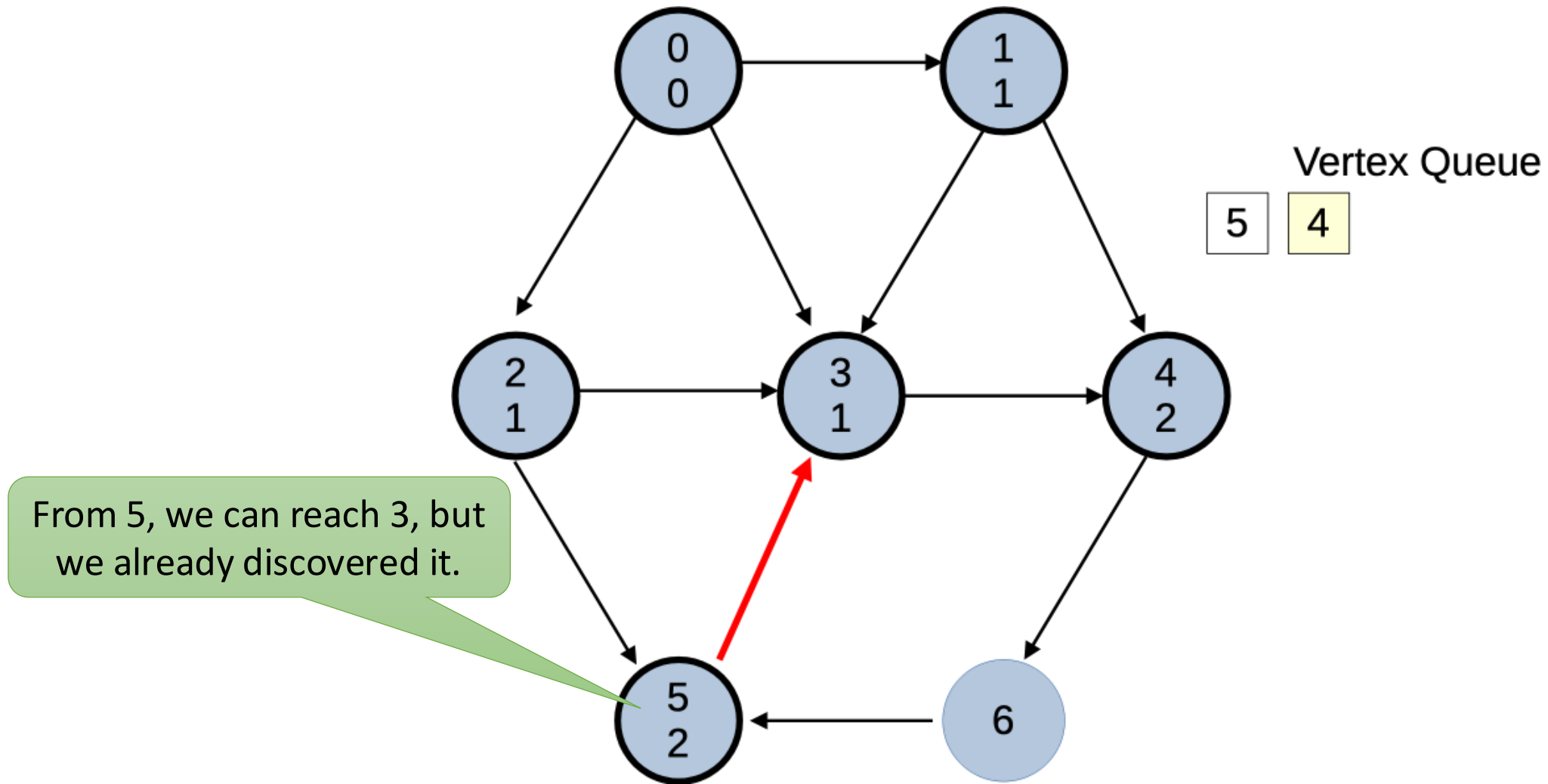


Breadth-First Order

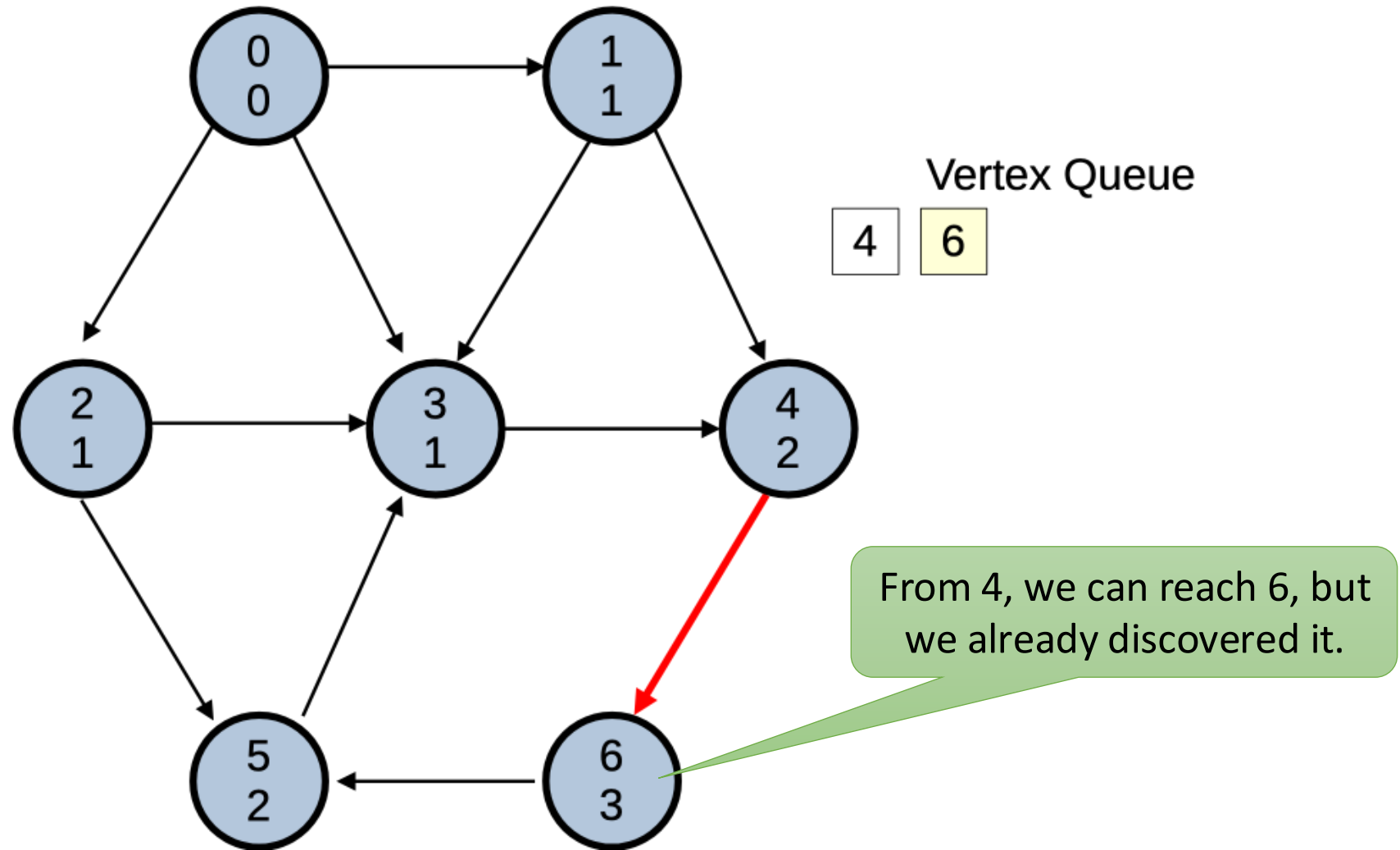


From 1, we can reach 4, but we already discovered it.

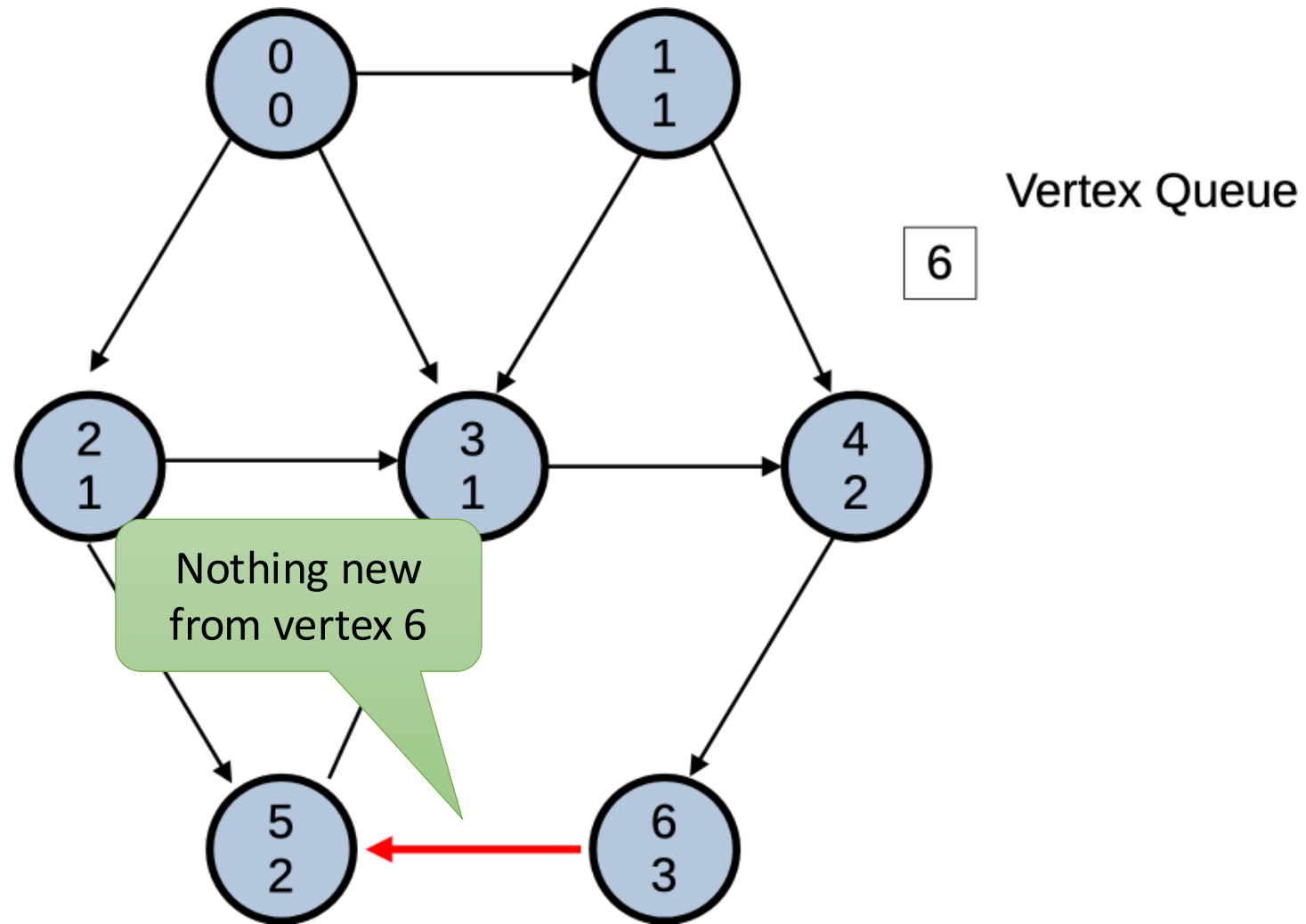
Breadth-First Order



Breadth-First Order



Breadth-First Order



Breadth-First Traversal

```
distance = [ -1, -1, -1 ... -1 ];    // Distance to each vertex we reach.

queue< int > Q;

Q.add_back( 0 );                      // Start at vertex 0
distance[ 0 ] = 0;

while ( Q.size() ) {
    int i = Q.remove_front();
    for ( j : neighbors[ i ] )
        if ( distance[ j ] < 0 ) {    // First time we've found this vertex?
            distance[ j ] = distance[ i ] + 1; // One hop farther than i.
            Q.add_back( j );           // Explore from j (later)
        }
}
```

Multiple starting points

